Insight-to-Action Engine

Production-Grade Architecture & Implementation Guide

---

1. System Overview

The **Insight-to-Action Engine** is an enterprise-grade AI platform that transforms raw user inputs (PDFs, URLs, text) into actionable business recommendations through a sophisticated multi-agent pipeline orchestrated via Google Vertex AI Agent Builder.

Core Capabilities

- **Parse & normalize** diverse input formats (PDF, URL, plain text)
- **Extract non-trivial insights** with evidence citations using Gemini Pro
- **Quantify real-world impact** and generate urgency scores
- **Generate 3 ranked actions** with confidence metrics and parameters
- **Simulate execution** with mock APIs returning before/after state transitions

Key Features

- Real-time JSON-based data flow with strong typing
- Multi-platform support: iOS, Android (Flutter), Web (React/Next.js)
- Vertex AI Agent Builder orchestration (no agent-to-agent calls)
- Stateless execution for horizontal scalability
- Production-grade security with RBAC, prompt injection protection, sandboxing

---

2. High-Level Architecture

Architecture Overview

The system follows a service-oriented architecture with clear separation between frontends, stateless backend, and AI orchestration:

Layer	Technology	Responsibility
Frontend	Flutter + React/Next.js	User interaction, state management, real-time rendering
Backend API	FastAPI + Python 3.12	Route requests, validate inputs, orchestrate agents
AI Orchestration	Vertex AI Agent Builder	Manage 5-agent pipeline, tool routing, retry logic
Agents	Gemini Pro + Tools	Ingestion, extraction, analysis, generation, simulation
Infrastructure	Cloud Run + Docker	Compute, networking, load balancing, auto-scaling
Data Storage	PostgreSQL + Redis	Persistent user data, session caching

Request Flow

```
User → Mobile/Web App → FastAPI Backend → Vertex AI Orchestrator
→ 5 Agents (Ingestion, Insight, Impact, Action, Simulation)
→ PostgreSQL/Cache → Response
```

Service Communication

- **REST API:** Frontend ↔ Backend (JSON over HTTPS)
- **gRPC:** Backend ↔ Vertex AI (internal, same VPC)
- **WebSocket:** Real-time updates (agent progress, simulation status)

- **Pub/Sub**: Event streaming (audit logs, notifications)

Cloud Infrastructure

- **Cloud Run**: Auto-scaling containerized FastAPI
- **Cloud SQL**: Managed PostgreSQL (high availability)
- **Cloud Memorystore**: Managed Redis for caching
- **Cloud Storage**: User file uploads with signed URLs
- **Cloud Logging & Trace**: Observability
- **Cloud Load Balancing**: Global traffic distribution

---

3. Monorepo Project Structure

```
insight-to-action/
├── apps/
│   ├── backend/
│   │   ├── app/
│   │   │   ├── routers/          # API endpoints
│   │   │   ├── services/         # Business logic
│   │   │   ├── models/           # SQLAlchemy ORM
│   │   │   ├── middleware/       # Auth, logging, rate limiting
│   │   │   ├── utils/            # Helpers
│   │   │   ├── config.py         # Environment config
│   │   │   └── main.py           # FastAPI app
│   │   ├── tests/
│   │   │   ├── unit/
│   │   │   └── integration/
│   │   ├── requirements.txt
│   │   └── Dockerfile
│   ├── frontend-web/
```

```
│   │   ├── src/
│   │   │   ├── components/      # Reusable UI components
│   │   │   ├── pages/           # Next.js routes
│   │   │   ├── hooks/           # Custom hooks
│   │   │   ├── services/        # API client
│   │   │   ├── store/           # Zustand state
│   │   │   └── styles/
│   │   ├── next.config.js
│   │   └── package.json
│   └── frontend-mobile/
│       ├── lib/
│       │   ├── models/          # Data classes (freezed)
│       │   ├── screens/         # Full-screen pages
│       │   ├── widgets/         # Reusable components
│       │   ├── services/        # API client
│       │   └── providers/       # Riverpod providers
│       ├── pubspec.yaml
│       └── android/ / ios/
├── packages/
│   ├── schemas/                 # Shared JSON schemas (TypeScript/Pydantic)
│   ├── api-client/              # OpenAPI-generated client
│   └── utils/                   # Shared utilities
├── infra/
│   ├── terraform/               # GCP resources
│   ├── kubernetes/              # K8s manifests
│   ├── docker/                  # Dockerfiles
│   └── ansible/                 # Deployment automation
├── .github/
│   └── workflows/               # GitHub Actions CI/CD
├── docs/
│   ├── API.md
│   ├── ARCHITECTURE.md
│   └── DEPLOYMENT.md
```

```
├── docker-compose.yml          # Local development
├── pyproject.toml              # Python metadata
├── package.json                # Workspace root
└── README.md
```

---

4. Backend Architecture (FastAPI)

Core Design Principles

- **Stateless**: No in-memory state; all state in PostgreSQL/Redis
- **Async-first**: All I/O operations are non-blocking
- **Middleware-driven**: Cross-cutting concerns handled at middleware layer
- **Service-oriented**: Business logic isolated in reusable services
- **Contract-driven**: API contracts defined via Pydantic models

FastAPI Application Structure

Main Application (`app/main.py`)

Entry point that initializes FastAPI, configures middleware, includes routers, sets up CORS, security, and exception handlers.

Routers (`app/routers/`)

- `auth.py`: Login, signup, token refresh, logout
- `documents.py`: Upload, list, delete user documents
- `insights.py`: Submit for analysis, retrieve results
- `actions.py`: List ranked actions, confirm action, simulate
- `websocket.py`: Real-time progress updates

Services (`app/services/`)

- `auth_service.py`: User authentication, token generation, password hashing
- `document_service.py`: File upload, parsing, storage
- `orchestration_service.py`: Calls Vertex AI Agent Builder
- `cache_service.py`: Redis operations for rate limiting, session caching

Models (`app/models/`)

- `user.py`: SQLAlchemy User table definition
- `document.py`: Document metadata and relationships
- `insight.py`: Insight records with citations
- `action_execution.py`: Action history and simulation results

Middleware (`app/middleware/`)

- `auth.py`: JWT validation and user context injection
- `logging.py`: Structured logging with correlation IDs
- `rate_limiting.py`: Token bucket algorithm per user
- `error_handling.py`: Global exception handler with sanitized responses

Async Request Lifecycle

1. Client sends request
2. Rate limiting middleware checks quota
3. Auth middleware validates JWT and injects user context
4. Router calls service method (async)
5. Service makes database query (async via SQLAlchemy AsyncSession)
6. Service calls Vertex AI (async via httpx.AsyncClient)
7. Response is serialized and returned

8. Logging middleware records request metadata

WebSocket Support

Enable real-time updates for agent execution progress:

- Client connects to `/ws/{request_id}`
- Backend polls Vertex AI for status via background task
- When status changes, broadcast event to connected WebSocket
- Close connection when request completes

Error Handling Strategy

Error Type	HTTP Code	Client Action
Invalid input	400	Display validation errors
Unauthorized	401	Redirect to login
Forbidden	403	Show access denied
Rate limited	429	Exponential backoff + retry
Agent timeout	504	Retry with longer timeout
Internal error	500	Show generic error + request ID

---

5. Agent Orchestration Design

The 5-Agent Pipeline

All agent calls routed through single Vertex AI Agent Builder orchestrator for centralized

control:

Agent	Role	Input	Output
Ingestion	Parse & clean	Raw PDF/URL/text	InputDocument
Insight	Extract insights	InputDocument	InsightResult[]
Impact	Quantify impact	InsightResult	ImpactResult
Action	Generate actions	ImpactResult	ActionItem[]
Simulation	Execute & mock	ActionItem + context	SimulationResult

Orchestrator Design: Tool-Based Architecture

Each agent exposed as a "tool" to orchestrator. Orchestrator decides which tool to call based on pipeline stage.

Tool Definitions

Tool Name	Purpose	Signature
ingest_document	Parse PDF, URL, or text	`(input: string, type: enum) → InputDocument`
extract_insights	Find key insights with evidence	`(document: InputDocument) → InsightResult[]`
analyze_impact	Quantify consequences	`(insight: InsightResult) → ImpactResult`
generate_actions	Create 3 ranked actions	`(impact: ImpactResult) → ActionItem[]`
simulate_execution	Mock action execution	`(action: ActionItem) → SimulationResult`

Orchestration Logic

1. Backend receives user input
2. Creates pipeline execution record in PostgreSQL
3. Calls Vertex AI Orchestrator with ingest_document tool
4. Orchestrator calls Ingestion Agent → returns InputDocument
5. Orchestrator calls extract_insights → returns InsightResult[]
6. For each insight, orchestrator calls analyze_impact → returns ImpactResult
7. Orchestrator calls generate_actions → returns ActionItem[]
8. Store results in PostgreSQL, notify client via WebSocket
9. User confirms action → backend calls orchestrator with simulate_execution

Retry & Failure Handling

- **Exponential backoff**: 1s, 2s, 4s, 8s, 16s (max 3 retries)
- **Timeouts**: 30s for parsing, 60s for LLM calls
- **Circuit breaker**: If 5 consecutive failures, mark agent unhealthy
- **Graceful degradation**: Return cached results if available

Prompt Injection Protection

- **Input validation**: Regex filters for known attack patterns
- **Prompt sandboxing**: System prompts immutable, user input appended
- **Output validation**: Parse LLM JSON response strictly
- **Rate limiting**: Max 10 requests/minute per user

Validation Layers

Pre-Agent Validation: Pydantic models validate input (non-empty content, valid URLs, proper PDF structures)

Post-Agent Validation: Strict JSON schema validation against expected output types

Business Logic Validation:

- Insights must have confidence > 0.5
- Actions ranked with confidence sum = 1.0
- Impact score between 0-100

---

6. API Contracts & Schemas

JSON Schema Definitions

InputDocument

json

```
{
  "id": "uuid",
  "content": "string",
  "source_type": "pdf|url|text",
  "source_url": "string|null",
  "uploaded_at": "timestamp",
  "file_hash": "sha256",
  "extraction_metadata": {
    "page_count": "int",
    "language": "string",
    "encoding": "string"
  }
}
```

InsightResult

json

```
{
  "id": "uuid",
  "insight": "string",
  "category": "financial|operational|risk|opportunity",
  "confidence": "0.0-1.0",
  "evidence": [
    {
      "citation": "string",
      "page_number": "int",
      "relevance_score": "0.0-1.0"
    }
  ],
  "extracted_at": "timestamp"
}
```

ImpactResult

json

```
{
  "id": "uuid",
  "insight_id": "uuid",
  "impact_score": "0-100",
  "urgency_score": "0-100",
  "affected_areas": ["string"],
  "quantified_impact": {
    "metric": "string",
    "current_value": "number",
    "projected_value": "number",
    "unit": "string"
  },
  "analysis_at": "timestamp"
}
```

ActionItem

json

```
{
  "id": "uuid",
  "rank": "1|2|3",
  "title": "string",
  "description": "string",
  "confidence": "0.0-1.0",
  "parameters": {},
  "estimated_effort": "hours",
  "expected_roi": "percentage",
  "generated_at": "timestamp"
}
```

SimulationResult

```json
{
  "id": "uuid",
  "action_id": "uuid",
  "status": "success|partial|failed",
  "before_state": {},
  "after_state": {},
  "execution_logs": ["string"],
  "generated_email": "string|null",
  "simulated_at": "timestamp"
}
```

REST API Endpoints

Endpoint	Method	Auth	Purpose
/auth/signup	POST	None	Create account
/auth/login	POST	None	Get JWT token
/auth/refresh	POST	Refresh token	Renew access token
/documents	POST	Bearer	Upload document
/documents	GET	Bearer	List user docs
/documents/{id}	DELETE	Bearer	Delete document
/insights	POST	Bearer	Submit for analysis
/insights/{request_id}	GET	Bearer	Retrieve results
/actions/{insight_id}	GET	Bearer	Get ranked actions

Endpoint	Method	Auth	Purpose
/actions/{action_id}/simulate	POST	Bearer	Simulate action
/ws/{request_id}	WS	Bearer	Real-time updates

7. Database Schema

Core Tables

users

```
id (PK, UUID) | email (UNIQUE) | password_hash | first_name | last_name |
created_at | updated_at | is_active | subscription_tier
```

documents

```
id (PK, UUID) | user_id (FK) | filename | content_type | source_type
(pdf|url|text) |
source_url | file_size | file_hash | storage_path | uploaded_at | deleted_at
```

pipelines

```
id (PK, UUID) | user_id (FK) | document_id (FK) | status
(pending|processing|completed|failed) |
started_at | completed_at | error_message
```

insights

```
id (PK, UUID) | pipeline_id (FK) | insight_text | category | confidence |
evidence_json | created_at
```

impact_analyses

```
id (PK, UUID) | insight_id (FK) | impact_score | urgency_score |
affected_areas_json | quantified_impact_json | analyzed_at
```

actions

```
id (PK, UUID) | impact_id (FK) | rank | title | description | confidence |
parameters_json | estimated_effort | expected_roi | generated_at
```

simulations

```
id (PK, UUID) | action_id (FK) | status | before_state_json | after_state_json |
execution_logs_json | generated_email | simulated_at
```

audit_logs

```
id (PK, UUID) | user_id (FK) | action (upload|analyze|simulate|delete) |
resource_id | resource_type | timestamp | ip_address | user_agent
```

Indexes

- `documents (user_id, created_at DESC)`
- `insights (pipeline_id, confidence DESC)`
- `actions (impact_id, rank)`
- `audit_logs (user_id, timestamp DESC)`

8. Frontend Architecture

Web (React/Next.js)

State Management

- **Zustand**: Global state (auth, user, notifications)
- **React Query**: Server state (documents, insights, actions)
- **Context API**: Theme and localization

File Organization

- `src/pages/`: Next.js file-based routing
- `src/components/`: Reusable UI components
- `src/hooks/`: Custom hooks (useAuth, useDocument, usePipeline)
- `src/services/`: API client (axios with interceptors)
- `src/store/`: Zustand stores

Key Pages

- `/login`: Authentication
- `/dashboard`: Main workspace
- `/documents`: Upload & manage files
- `/insights`: View analysis results
- `/actions`: Ranked recommendations & simulation

Real-Time Updates

- WebSocket connection to `/ws/{request_id}`

- Progress bar shows pipeline stages
- Toast notifications for errors/completions

Mobile (Flutter)

State Management

- **Provider**: Dependency injection
- **Riverpod**: Reactive state (planned upgrade)

File Organization

- `lib/models/`: Data classes (freezed)
- `lib/screens/`: Full-screen pages
- `lib/widgets/`: Reusable components
- `lib/services/`: API client, local storage
- `lib/providers/`: Riverpod providers

Navigation

- **GoRouter**: Declarative routing
- Deep linking support for direct document access

GCP Services Overview

Service	Purpose	Config	Cost Optimization
Cloud Run	Backend API	2 CPU, 2 GB RAM, 300s timeout	Min 1, max 10 instances
Cloud SQL	PostgreSQL 15	db-custom-4-16GB (HA, multi-region)	Scheduled backups, CUD
Memorystore	Redis cache	2GB standard tier	Auto-eviction, TTL expiry
Cloud Storage	User uploads	Multi-region bucket, lifecycle	Move to Coldline after 30 days
Vertex AI	Agent orchestration	Agents API, Gemini Pro	Token-based pricing

Docker & Containerization

Dockerfile: Multi-stage build (builder → runtime). Base image: `python:3.12-slim`

Image Registry: Artifact Registry with vulnerability scanning and Binary Authorization

CI/CD Pipeline (GitHub Actions)

1. **Test**: Run unit tests, lint, type-check
2. **Security**: SAST (Bandit), dependency audit
3. **Build**: Build Docker image, push to Artifact Registry
4. **Deploy**: Deploy to Cloud Run (staging), run integration tests
5. **Production**: Manual approval, then deploy to prod Cloud Run

6. **Monitoring**: Set up Cloud Monitoring alerts

Secrets Management

- **Google Secret Manager**: Store database passwords, API keys, JWT secrets
- **Cloud Run integration**: Secrets auto-mounted as environment variables
- **Rotation**: Automated key rotation for sensitive secrets

Scaling Strategy

Horizontal Scaling

- Cloud Run: Auto-scales 1-10 instances based on request rate
- Load Balancer: Distributes traffic across instances

Vertical Scaling

- Cloud SQL: Upgrade machine size if CPU/memory bottleneck
- Memorystore: Increase cache size for hot keys

Caching

- **Session cache (Redis)**: Auth tokens, 1 hour TTL
- **Query cache (Redis)**: Frequent insights, 24 hour TTL
- **CDN**: Static assets, reduce origin load

Monitoring & Observability

Cloud Logging

- Structured logs (JSON) with correlation IDs
- Log Router: ERROR/CRITICAL → alert
- Log Insights: Analyze patterns, find anomalies

Cloud Trace

- Distributed tracing: Track request end-to-end
- Identify bottlenecks: Database, API, agent latency

Cloud Monitoring

- Key metrics: Request latency (p50/p99), error rate, CPU/memory
- Alerts: Error rate > 5% or p99 latency > 5s
- Dashboards: Real-time system health

Disaster Recovery

- **Backups**: Automated daily, retained 30 days
- **Point-in-time recovery**: Last 7 days
- **Failover**: High availability mode (2 replicas, different zones)
- **Runbook**: Incident response procedures

---

10. Security Architecture

API Security

Authentication

- **JWT**: Access token (15 min) + refresh token (7 days)
- **Storage**: httpOnly cookie (XSS protection)
- **Password**: bcrypt with salt (cost 12)

Authorization

- **RBAC**: user, admin, data_analyst roles
- **Resource-level checks**: Only access own data
- **Scope-based**: Rate limits vary by subscription

HTTPS & TLS

- Enforce HTTPS: Redirect HTTP → HTTPS
- TLS 1.3: Minimum, managed by Cloud Load Balancer
- HSTS: max-age=31536000, includeSubDomains

Prompt Injection Protection

- **Input sanitization**: Remove control chars, filter attack patterns
- **Prompt isolation**: System prompts read-only, user input appended
- **Output validation**: Strict JSON schema
- **Rate limiting**: Max 10 requests/minute per user

File Upload Security

- **Type validation**: Only PDF, URLs, text
- **File size limit**: Max 50 MB
- **Virus scan**: Cloud DLP for malware detection
- **Storage isolation**: Private GCS bucket with signed URLs

Data Protection

- **Encryption at rest**: AES-256 via Cloud SQL & GCS
- **Encryption in transit**: TLS 1.3
- **Data retention**: Delete after 30 days of account closure
- **PII redaction**: Mask sensitive data in logs

Rate Limiting

- **Sliding window (Redis)**: 100 requests/min per user
- **Adaptive limits**: Premium users 500 requests/min
- **Retry-After header**: Tell client when to retry

Audit & Compliance

- **Audit logs**: User actions (upload, analyze, delete)
- **Immutable logging**: Cloud Logging → Cloud Storage
- **Data residency**: Authorized regions only

---

11. DevOps & CI/CD

Local Development

Setup

- Clone monorepo
- Install Python 3.12+
- Install Node.js 18+
- Install Docker & Docker Compose
- Copy .env.example → .env

Docker Compose

Services: PostgreSQL, Redis, Vertex AI emulator, FastAPI backend

Testing Strategy

Unit Tests

- **pytest** for backend (test/unit/)
- **Jest** for frontend (src/**tests**/)
- Target: 80% code coverage

Integration Tests

- Test API endpoints with mock agents
- testcontainers for PostgreSQL + Redis
- Target: 60% coverage

End-to-End Tests

- **Playwright** for web UI
- **Appium** for mobile
- Smoke tests in staging

Code Quality

Linting & Formatting

- **ruff**: Python linter
- **black**: Python formatter
- **eslint** + **prettier**: JavaScript/TypeScript
- Pre-commit hooks: Run before commit

Type Checking

- **mypy**: Python static type checking
- **TypeScript**: React/Next.js frontend

Git Workflow

- **Branching**: main (prod), staging, feature/*
- **Commits**: Conventional commits (feat:, fix:, docs:)
- **PR reviews**: At least 1 approval
- **Protect main**: Require tests to pass, no direct pushes

Environment Management

- **Development**: Local .env with test credentials
- **Staging**: GCP staging project, real Vertex AI
- **Production**: GCP production project, secrets from Secret Manager

---

12. Implementation Roadmap

Phase 1: MVP (Weeks 1-8)

Deliverables

- FastAPI backend with user auth (signup/login)
- Simple document upload (PDF only)
- Basic orchestrator calling Ingestion + Insight agents
- PostgreSQL schema (users, documents, insights)
- React web dashboard (login, upload, view results)
- Deploy to Cloud Run (staging)

Team (3)

- 1 Backend engineer
- 1 Frontend engineer

- 1 AI/Prompt engineer

Phase 2: Full Pipeline (Weeks 9-16)

Deliverables

- Complete 5-agent orchestration (Impact, Action, Simulation)
- URL & text input support
- WebSocket real-time progress
- Redis caching & rate limiting
- Flutter mobile app (basic features)
- Comprehensive test suite (80% coverage)
- API documentation (OpenAPI/Swagger)

Team (5)

- 2 Backend engineers
- 1 Frontend engineer
- 1 Mobile engineer
- 1 AI/Prompt engineer

Phase 3: Production & Scale (Weeks 17-24)

Deliverables

- Production deployment (multi-region, HA)
- Advanced security (RBAC, audit logs, DLP)
- Monitoring & alerting (Cloud Monitoring)
- Disaster recovery & backups
- Subscription tier system
- Advanced analytics dashboard

- Team onboarding & runbooks

Team (7)

- 2 Backend engineers
- 1 DevOps/SRE engineer
- 1 Frontend engineer
- 1 Mobile engineer
- 1 QA engineer

Timeline

Phase	Timeline	Team	Key Milestones
MVP	Weeks 1-8	3	Auth, Upload, Analysis, Web Deploy
Full Pipeline	Weeks 9-16	5	5 agents, Mobile, Cache, Tests
Production	Weeks 17-24	7	HA, Security, Monitoring, Launch

---

13. Scaling Recommendations

Performance Optimization

Database

- **Connection pooling**: PgBouncer for efficient connections
- **Read replicas**: Route SELECT queries to replicas
- **Partitioning**: Partition insights table by month

API

- **Pagination**: Limit 100 documents per page
- **Response compression**: gzip for large payloads
- **Async processing**: Cloud Tasks for heavy operations

Agent Pipeline

- **Batch processing**: Queue multiple insights for parallel analysis
- **Result caching**: Store agent outputs in Redis (24h TTL)
- **Model selection**: Gemini 1.5 Flash for fast, cheap completions

Cost Optimization

- **Committed use discounts**: Save 25-40% on compute
- **Spot VMs**: For non-critical workloads
- **Cloud Storage lifecycle**: Move to Coldline after 30 days
- **Model optimization**: Batch API calls to reduce token costs

Capacity Planning

- **Estimate**: 1,000 users × 10 insights/user/month = 10K insights/month
- **Storage**: ~1GB/1K insights (~100KB per insight)
- **Agent costs**: ~$0.01 per Gemini call
- **Monthly cost target**: <$1000 MVP, <$5000 scale

Event-Driven Architecture

- **Pub/Sub topics**: document.uploaded, insight.generated, action.simulated
- **Subscribers**: Analytics pipeline, notification service, audit logger
- **Benefits**: Decoupling, real-time updates, scalability

Failure Recovery

- **Agent timeout**: Retry with backoff, return cached result
- **Database loss**: Circuit breaker + backoff, queue to Pub/Sub
- **Cache miss**: Rebuild from database on demand

Next Steps

1. **Review** architecture with team and stakeholders
2. **Set up** version control and CI/CD pipeline
3. **Create** project tickets for Phase 1 milestones
4. **Begin** development on MVP backend
5. **Iterate** on agent prompts with sample documents

End of Architecture Document

For detailed questions, refer to specific sections or reach out to the technical lead.